

Arrays: Storing and Processing Data

Matteo Fasiolo

So far we dealt with variables holding individual values:

```
int x = 7;
```

```
double y = 1.1;
```

Now we will see how to use **arrays** to store several values.

By the end of the lecture, you should be able to:

1. declare and initialise a **fixed-size array**;
2. read and write elements of arrays;
3. process an array using a loop;
4. recognise an out-of-bounds access;
5. describe the conceptual memory layout of an array.

Why do we need arrays?

Data scientists rarely work with only one number.

Examples include:

- temperatures recorded each hour;
- marks for all students in a class;
- the features describing one observation;
- parameters of a statistical model.

We need **one variable that can hold a sequence of values**.

Can't we just use many scalar variables?

Consider the problem of adding two vectors:

$$a = [1, 2, 3], \quad b = [2, 3, 4], \quad c = a + b.$$

Without arrays:

```
double a0 = 1.0, a1 = 2.0, a2 = 3.0;  
double b0 = 2.0, b1 = 3.0, b2 = 4.0;  
  
double c0 = a0 + b0;  
double c1 = a1 + b1;  
double c2 = a2 + b2;
```

What if the vectors have 100 or 10,000 elements?

Introducing arrays

An **array** stores a **fixed-length** sequence of elements of the **same type**.

```
double a[3];
```

This declaration reserves space for three double elements:

name	a		
elements type	double		
indices	0	1	2
elements	a[0]	a[1]	a[2]

The length is 3, but the last valid index is 2.

Declare and initialise

Declare an array and provide all its initial values:

```
double a[3] = {1.0, 2.0, 3.0};
```

NOTE: the declared length should match the length of the list.

If values are provided, C can infer the length:

```
double b[] = {2.0, 3.0, 4.0};
```

Both a and b have length 3.

Initialise before reading

This declaration does **not** initialise the elements:

```
double result[3];
```

So the value of the elements is arbitrary.

Do not read an element before giving it a value:

```
result[0] = 3.0;  
result[1] = 5.0;  
result[2] = 7.0;
```

Zero-based indexing

For an array of length n , the valid indices are

$$0, 1, \dots, n - 1.$$

```
double a[] = {1.0, 2.0, 3.0};

printf("%f\n", a[0]); // first element: 1.0
printf("%f\n", a[2]); // third element: 3.0

a[1] = 10.0;           // change the second element
```

`a[i]` means “the element at index i ”.

The index can be a variable

```
double a[] = {1.0, 2.0, 3.0};  
int i = 1;  
  
printf("%f\n", a[i]);
```

This prints 2.0.

Changing `i` changes which element is accessed.

This is what allows a loop to process the whole array.

Quick check: valid or invalid?

Suppose `double x[5];` has been declared.

Expression	Valid?
<code>x[0]</code>	?
<code>x[4]</code>	?
<code>x[5]</code>	?
<code>x[-1]</code>	?
<code>x[i]</code> , where <code>i == 3</code>	?

Valid indices are 0, 1, 2, 3 and 4.

Arrays and for loops

Use the loop counter as the index:

```
double a[] = {1.0, 2.0, 3.0};  
int length = 3;  
  
for (int i = 0; i < length; i = i + 1) {  
    printf("a[%d] = %f \n", i, a[i]);  
}
```

The condition `i < length` makes the final index `length - 1`.

Back to vector addition

```
double a[] = {1.0, 2.0, 3.0};  
double b[] = {2.0, 3.0, 4.0};  
double c[3];  
int length = 3;  
  
for (int i = 0; i < length; i = i + 1) {  
    c[i] = a[i] + b[i];  
}
```

After the loop:

```
c = [3.0, 5.0, 7.0]
```

Switch to VS Code

Open `array_basics.c`

The debugger confirmed:

- one array name refers to several elements;
- `i` selects the element used in an iteration;
- array elements can be inspected and changed separately;
- `i < length` keeps every access within the valid range.

The length is part of the algorithm

An array and its length must agree with every loop that processes it.

```
double a[] = {1.0, 2.0, 3.0};  
int length = 3;  
  
for (int i = 0; i < length; i = i + 1) {  
    // process a[i]  
}
```

The array does not automatically stop a loop whose bound is wrong.

Out-of-bounds access

```
double a[ ] = {1.0, 2.0, 3.0};  
  
printf("%.1f\n", a[3]);
```

The valid indices are 0, 1 and 2.

Therefore a[3] is out of bounds.

C normally does not check array bounds for us.

Undefined behaviour

An out-of-bounds access has **undefined behaviour**.

The program might:

- crash (*segfault*);
- print an arbitrary or plausible-looking value;
- overwrite unrelated data;
- behave differently after a small code or compiler change.

You can not trust the output of your program in this situation.

Switch to VS Code

Open `array_bounds.c`

Summary: Errors and bugs

Things get worse from top to bottom:

Category	Example
Compilation error	Missing ; or incompatible syntax
Runtime failure	A program crashes or never finishes
Undefined behaviour	Accessing outside an array
Logic error	Valid program calculates the wrong answer

Successful compilation does not prove that a program is correct!

How are arrays stored in memory?

Array elements occupy a **contiguous sequence of memory locations**.

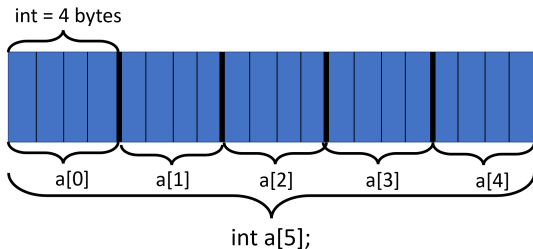
index	0	1	2
value	7.2	2.4	1.5

- every element has the same type and size;
- neighbouring indices correspond to neighbouring elements;

Example: let's consider an integer array

```
int a[5];
```

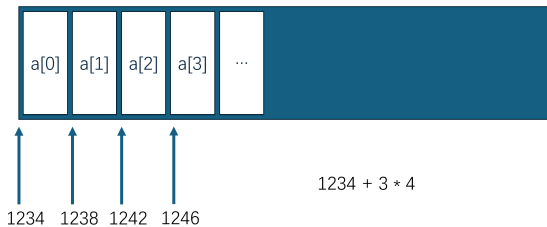
Recall, an integer variable takes up 4 bytes of memory.



Suppose that each byte has its own **address**.

The first integer is stored at memory address 1234.

What is the memory address of `a[3]`?



Check your understanding

Suppose `double data[10];` is declared.

1. What are the first and last valid indices?
2. If `a[0]` has address 1000, what is the address of `a[2]`?
3. How many times does this loop run for?

```
for (int i = 0; i < 10; i = i + 1) {  
    data[i] = 0.0;  
}
```

4. Why would changing `<` to `<=` introduce a bug?

Revisiting computational complexity

Consider a C program implementing vector addition

```
for (int i = 0; i < n; i = i + 1) {  
    c[i] = a[i] + b[i];  
}
```

where n is the length of a , b and c .

What is the computational complexity of vector addition $O(??)$?

We have that:

- the loop runs n times;
- each iteration performs a fixed amount of work:
 - 3 memory accesses
 - 1 addition
- the time complexity is $O(n)$.

Doubling the vector length approximately doubles the work.

Another example, assume that:

- a and b are arrays of doubles
- n is the length of arrays a and b
- p is a positive integer

What is this code doing?

```
for (int i = 0; i < n; i = i + 1) {  
  
    b[i] = 1.0;  
  
    for (int k = 0; k < p; k = k + 1)  
  
        b[i] = b[i] * a[i];  
  
    }  
}
```

```
for (int i = 0; i < n; i = i + 1) {  
    b[i] = 1.0;  
    for (int k = 0; k < p; k = k + 1)  
        b[i] = b[i] * a[i];  
}  
}
```

It is computing the p -th power of $a[i]$, and storing it in $b[i]$.

What is its time-complexity?

It should take into account both n and p .

```
for (int i = 0; i < n; i = i + 1) {  
    b[i] = 1.0;  
    for (int k = 0; k < p; k = k + 1)  
        b[i] = b[i] * a[i];  
    }  
}
```

The bottleneck is $b[i] = b[i] * a[i];$.

We run it p times, for each of the n elements.

So, the cost is $O(np)$.

Assume computing 5th power of array of length 10 takes 1s.

How long will it take with $p = 10$ and $n = 20$ (approximately)?

Conclusion

1. An array stores a fixed-length sequence of elements of one type.
2. An array of length n has indices 0 to $n-1$.
3. A loop can process each element using its counter as an index.
4. C normally does not detect out-of-bounds access.
5. Array elements occupy contiguous memory.